

# DI REFACTOR

## IMPLEMENTATION & REFACTOR SPECIFICATION

Core Library — *Dependency Injection Migration*

Version 2.0 — Enhanced Edition

### v2 Enhancements

This document extends the original v1 spec with: constructor selection policy, IDisposable teardown, scope lifecycle, thread-safety hardening, aggregate error reporting, migration rollback strategy, Editor diagnostics, boot-time performance budgeting, and a comprehensive test matrix.

## 1. Executive Summary

The Core library currently implements a Service Locator pattern, not true Dependency Injection. While the ServiceLocator is functional, it has fundamental structural problems that make the system fragile, untestable, and impossible to verify at boot time.

This document specifies the full migration to real constructor-based DI in five phases. Each phase is independently deployable, keeping the codebase functional throughout the migration.

### KEY OUTCOME

After this migration: every service is fully wired at construction, the container validates the entire dependency graph at boot, a missing registration produces a clear error at startup — not a `NullReferenceException` deep in gameplay — and every service is properly disposed when its scope ends.

## 2. Gap Analysis — What the v1 Spec Missed

The v1 spec addresses the critical runtime bugs and the core registration migration. The following gaps were identified through secondary analysis and must be resolved in parallel with or after the v1 phases.

| Category           | Gap / Enhancement                                                                                                            | Why It Matters                                                                  | Addressed In   |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------|
| Category           | Gap / Enhancement                                                                                                            | Why It Matters                                                                  | Addressed In   |
| Constructor Policy | No rule for ambiguous ctors — <code>DValidator</code> uses 'longest ctor' heuristic silently                                 | Wrong ctor silently selected; breaks after refactors                            | §3.1 + Phase 2 |
| Teardown           | <code>IDisposable</code> never called on services — scopes <code>Dispose()</code> but <code>ServiceDescriptor</code> doesn't | Resource leaks ( <code>AudioSources</code> , file handles, network connections) | §3.2 + Phase 1 |

| Category                      | Gap / Enhancement                                                                                 | Why It Matters                                                                  | Addressed In                    |
|-------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------|
| <b>Scope Lifecycle</b>        | SceneServiceScope.Dispose() path not shown; no guaranteed call site                               | Scoped services leak between scene loads if scene unload doesn't call Dispose() | <a href="#">§3.3 + Phase 3</a>  |
| <b>Thread Safety</b>          | ConcurrentDictionary used for singleton cache but resolution lock not shown; double-init possible | Race condition: two threads both see null singleton and both construct it       | <a href="#">§3.4 + Phase 1</a>  |
| <b>Error Reporting</b>        | DIValidator throws on first error — fails fast but hides all other broken deps                    | Developer sees one error per boot instead of the full broken graph              | <a href="#">§3.5 + Phase 2</a>  |
| <b>Generic Types</b>          | Open generic registration (IRepository<T>) not addressed                                          | Forced to register every closed generic variant individually                    | <a href="#">§3.6 + Phase 2</a>  |
| <b>Circular Dep Detection</b> | Runtime stack check catches cycles but only at first Get() call — too late if validator passes    | Cycle involving lazy-resolved service bypasses boot validation                  | <a href="#">§3.7 + Phase 2</a>  |
| <b>Rollback Strategy</b>      | Phase 3 installer migration has no documented rollback path                                       | If a migrated installer breaks prod, no clear recovery path                     | <a href="#">§3.8</a>            |
| <b>Performance Budget</b>     | No boot-time performance targets or benchmarks defined                                            | DI overhead silent; startup regression undetected until frames drop             | <a href="#">§3.9</a>            |
| <b>Editor Diagnostics</b>     | No Unity Editor window for inspecting the registered service graph                                | Debugging registration issues requires log-diving                               | <a href="#">§3.10 + Phase 5</a> |
| <b>Test Infrastructure</b>    | No unit test scaffolding for the container itself                                                 | Bugs in ServiceLocator/DIValidator go undetected until runtime                  | <a href="#">§4</a>              |
| <b>Alias Double-Init</b>      | Alias pattern (Register<IFoo>(s => s.Get<Foo>())) can double-initialize if resolved independently | Two singleton instances of the same service                                     | <a href="#">§3.2</a>            |

## 3. Enhancements by Topic

### 3.1 Constructor Selection Policy

The v1 DIValidator picks the constructor with the most parameters. This is a silent heuristic with no escape hatch. Two problems arise: (a) a service with an internal convenience ctor (fewer params) gets the wrong one selected; (b) adding an overload changes selection behaviour without warning.

Resolution: add [PreferredConstructor] attribute. If present, the container and validator both use it. If absent, the 'most params' rule applies — but the validator emits a warning when more than one public constructor exists.

**BEFORE — silent heuristic**

**AFTER — explicit attribute**

```
// Validator silently picks this
// because it has more params
public AudioService(
    PoolService pool,
    SettingsService settings,
    ILoggerService log)

// This ctor is ignored
public AudioService(
    PoolService pool) { ... }
```

```
[PreferredConstructor]
public AudioService(
    PoolService pool,
    SettingsService settings,
    ILoggerService log)
{ ... }

// Validator warns if no
// [PreferredConstructor] and
// multiple ctors exist
```

New file: Utility/Attributes/PreferredConstructorAttribute.cs

```
[AttributeUsage(AttributeTargets.Constructor, AllowMultiple = false)]
public sealed class PreferredConstructorAttribute : Attribute { }
```

Update DIValidator to prefer [PreferredConstructor], warn on ambiguity:

```
var ctor = implType.GetConstructors()
    .FirstOrDefault(c => c.GetCustomAttribute<PreferredConstructorAttribute>() != null)
    ?? implType.GetConstructors().OrderByDescending(c => c.GetParameters().Length).First();

if (implType.GetConstructors().Length > 1 &&
    ctor.GetCustomAttribute<PreferredConstructorAttribute>() == null)
    warnings.Add($"{implType.Name} has {implType.GetConstructors().Length} public ctors" +
        " - add [PreferredConstructor] to make selection explicit.");
```

## 3.2 IDisposable Teardown & Scope Disposal

The v1 plan does not address teardown. Services holding unmanaged resources (audio, file handles, network connections) must be disposed when their scope ends. This is a correctness issue, not just a hygiene concern — in Unity, undisposed AudioSources and file handles accumulate across scene loads and cause hard-to-diagnose memory pressure.

### Rule

ServiceLocator and ServiceScope must call IDisposable.Dispose() on every tracked instance when the scope ends. Singletons are disposed when ServiceLocator.Shutdown() is called. Scoped instances are disposed when the SceneServiceScope is disposed.

Changes to ServiceScope.cs:

```
internal sealed class ServiceScope : IServiceScope, IDisposable
{
    private readonly ConcurrentDictionary<Type, object> _scopedInstances;
    private bool _disposed;

    public void Dispose()
    {
        if (_disposed) return;
        _disposed = true;
    }
}
```

```

    foreach (var instance in _scopedInstances.Values.OfType<IDisposable>())
    {
        try { instance.Dispose(); }
        catch (Exception ex)
        { Debug.LogError($"[ServiceScope] Dispose failed for {instance.GetType().Name}: {ex}"); }
    }
    _scopedInstances.Clear();
}
}

```

Changes to ServiceLocator.cs — add Shutdown():

```

public void Shutdown()
{
    // 1. Run IInitializableService.ShutdownAsync() for singletons
    // 2. Dispose all singleton instances that are IDisposable
    foreach (var (_, desc) in _descriptors)
    {
        if (desc.SingletonInstance is IDisposable d)
            try { d.Dispose(); }
            catch (Exception ex) { Debug.LogError($"..."); }
    }
    _descriptors.Clear();
    isInitialized = false;
}

```

Alias registrations (§3.2 gap) — the alias-via-lambda pattern is safe only if it forwards to an already-singleton-cached instance. The v1 alias approach is correct for that. Document the rule explicitly:

#### Alias Rule

An alias registration **MUST** use `scope.Get<ConcreteType>()` — not `new ConcreteType(...)`. The concrete type must already be registered as Singleton. This guarantees one instance regardless of which key resolves it.

## 3.3 SceneServiceScope Lifecycle Contract

The v1 spec adds `SceneServiceScope` but does not specify when it is created or when `Dispose()` is called. Without a clear contract, scoped services accumulate across scene loads.

| Event                | Action                                                                                           | Who Calls It                                              |
|----------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Scene load begins    | <code>SceneServiceScopeRegistry.CreateForScene(scene)</code>                                     | SceneLoader or Bootstrapper scene handler                 |
| Scene installer runs | scene-bound installer installs into the scope                                                    | Scene's <code>IInstaller.Install(sceneScope)</code>       |
| Scene active         | <code>SceneServiceScope</code> marked ready; <code>CanvasServiceInjector</code> resolves from it | <code>SceneServiceScopeRegistry.GetForScene(scene)</code> |

| Event               | Action                                                             | Who Calls It                                            |
|---------------------|--------------------------------------------------------------------|---------------------------------------------------------|
| Scene unload begins | SceneServiceScopeRegistry.DisposeForScene(scene) → scope.Dispose() | SceneLoader, before AsyncOperation.allowSceneActivation |
| Scene unloaded      | Verify registry no longer holds ref to disposed scope              | SceneLoader assertion / DValidator                      |

Add to SceneServiceScopeRegistry.cs:

```
public static void DisposeForScene(Scene scene)
{
    if (_scopes.TryRemove(scene.handle, out var scope))
        scope.Dispose();
    else
        Debug.LogWarning($"[SceneServiceScopeRegistry] No scope for '{scene.name}'");
}
```

### 3.4 Thread-Safety Hardening

The v1 spec introduces a ConcurrentDictionary for singleton caching but does not address the TOCTOU race on singleton instantiation. The pattern GetOrAdd(key, factory) is thread-safe for the dictionary mutation but the factory may be invoked multiple times concurrently — meaning two threads can both call new ServiceImpl() before either result is cached.

#### Risk

Unity's main thread constraint protects most singletons, but async UniTask work running off-thread (web requests, file I/O) can call Get<T>() concurrently. AudioService and SaveService are candidates.

Fix: use double-checked locking for Singleton construction:

```
private object GetOrCreateSingleton(ServiceDescriptor desc, IServiceScope scope)
{
    if (desc.SingletonInstance != null) return desc.SingletonInstance; // fast path
    lock (desc) // lock the descriptor itself – avoids global lock
    {
        if (desc.SingletonInstance != null) return desc.SingletonInstance; // won the race
        desc.SingletonInstance = desc.Factory(scope);
        TrackInitializable(desc.SingletonInstance);
        return desc.SingletonInstance;
    }
}
```

Additionally, \_resolutionStack must be thread-local, not a shared HashSet, to avoid cross-thread false circular-dependency detection:

```
// BEFORE – shared, unsafe
private readonly HashSet<Type> _resolutionStack = new();

// AFTER – one stack per thread
```

```
[ThreadStatic]
private static HashSet<Type> _resolutionStack;
private static HashSet<Type> ResolutionStack =>
    _resolutionStack ??= new HashSet<Type>();
```

### 3.5 Aggregate Error Reporting in DIValidator

The v1 DIValidator already collects all errors before throwing — that part is correct. The gap is in the Bootstrapper, which wraps each installer in a try/catch that re-throws immediately. This means a broken installer A prevents the validator from ever seeing installer B's errors.

Fix: accumulate installer failures and validate them alongside DI errors:

```
var installerErrors = new List<string>();
foreach (var installer in sorted)
{
    try { installer.Install(locator); }
    catch (Exception ex)
    { installerErrors.Add($"{installer.InstallerName}: {ex.Message}"); }
}

// Run DIValidator even if some installers failed
// - it will report missing deps caused by those failures
var validationPassed = false;
try { DIValidator.ValidateRegistrations(locator); validationPassed = true; }
catch (InvalidOperationException ex) { installerErrors.Add(ex.Message); }

if (installerErrors.Count > 0)
{
    var fullReport = string.Join("\n", installerErrors);
    Debug.LogError($"[Bootstrapper] {installerErrors.Count} boot error(s):\n{fullReport}");
    throw new BootstrapException(fullReport);
}
```

### 3.6 Open Generic Registration

The v1 spec has no mechanism for registering open generics such as IRepository<T>. Without it, every closed variant (IRepository<Enemy>, IRepository<Item>) must be registered individually — a maintenance burden that grows with the domain model.

Add RegisterOpenGeneric<TServiceOpen, TImplOpen>() to ServiceLocator:

```
// Registration
locator.RegisterOpenGeneric(typeof(IRepository<>), typeof(GenericRepository<>));

// Resolution - ServiceLocator.GetType() checks open-generic table
// when exact match not found:
if (!_descriptors.TryGetValue(serviceType, out var desc))
{
    if (serviceType.IsGenericType)
    {
```

```

var openDef = serviceType.GetGenericTypeDefinition();
if (_openGenerics.TryGetValue(openDef, out var openImpl))
{
    var closedImpl = openImpl.MakeGenericType(serviceType.GenericTypeArguments);
    Register(serviceType, closedImpl, lifetime); // auto-register closed type
    _descriptors.TryGetValue(serviceType, out desc);
}
}
}

```

## Scope

Open generic registration is a Phase 4 addition. It is not required for the v1 migration but should be in scope before any data-layer work that introduces repository patterns.

## 3.7 Static Circular Dependency Detection in DIValidator

The v1 ServiceLocator detects cycles at runtime via `_resolutionStack`. With `DIValidator`, we can detect them statically at boot before any service is resolved — catching cycles that would only appear in lazy-resolved paths.

Add a graph walk to `DIValidator.ValidateRegistrations()`:

```

// Build adjacency map: impl → [constructor dep types]
var edges = new Dictionary<Type, List<Type>>>();
foreach (var (svcType, desc) in locator.descriptors)
{
    if (desc.ImplementationType == null) continue;
    var ctor = GetPreferredConstructor(desc.ImplementationType);
    edges[desc.ImplementationType] = ctor.GetParameters()
        .Select(p => p.ParameterType).ToList();
}

// DFS cycle detection
foreach (var root in edges.Keys)
    if (HasCycle(root, edges, new HashSet<Type>(), new HashSet<Type>(), out var cycle))
        errors.Add($"Circular dependency: {string.Join(" → ", cycle)}");

```

## 3.8 Migration Rollback Strategy

The v1 plan migrates each installer independently, which is correct. The missing piece is a rollback mechanism for when a migrated installer causes a regression in production before the full phase is complete.

| Strategy                          | Details                                                                                                                                                                            |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Feature flag per installer</b> | Wrap each installer migration in a static bool (compile-time stripped in release). Allows hot-switching between old and new registration paths in QA builds without a full revert. |
| <b>Git branch strategy</b>        | Each Phase 3 installer migration is a separate PR. <code>DIValidator</code> is run in CI against every PR. A broken installer fails CI before merge, not in production.            |

|                                 |                                                                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dual-registration period</b> | During migration, keep old locator.Register(new ...) lines commented-out (not deleted) until DIValidator has been green for two consecutive release builds. Then remove. |
| <b>InstallationMode enum</b>    | Bootstrapper reads InstallationMode.Legacy or .DI from a build config asset. Legacy path calls old Install(); DI path calls new Install(). Remove Legacy after Phase 5.  |

## 3.9 Boot-Time Performance Budgeting

DI container startup overhead is dominated by reflection. Without benchmarks, startup regressions are invisible until they manifest as frame drops or load screen complaints.

### Target

Container startup (registration + validation + singleton init) must complete within 200 ms on the minimum supported device. InstallerSorter and DIValidator should each contribute < 5 ms on typical graph sizes (< 150 registrations).

Hardening measures:

- Cache all reflection in DIValidator using a static ConcurrentDictionary<Type, ConstructorInfo> — reflection cost paid once per type, not once per boot.
- ServiceInjector.\_fieldCache already planned in v1. Extend the same pattern to DIValidator's constructor introspection.
- InstallerSorter performs no reflection — it is O(V+E) on the installer graph, which is bounded and fast by construction.
- Add a Stopwatch block in Bootstrapper around the Sort → Install → Validate → Init sequence. Log each segment duration at Debug level. Throw if total exceeds budget in development builds.

Benchmark logging addition to Bootstrapper:

```
#if UNITY_EDITOR || DEVELOPMENT_BUILD
var sw = System.Diagnostics.Stopwatch.StartNew();
sorted.ForEach(i => i.Install(locator));
Debug.Log($"[Boot] Install: {sw.ElapsedMilliseconds}ms");
sw.Restart();
DIValidator.ValidateRegistrations(locator);
Debug.Log($"[Boot] Validate: {sw.ElapsedMilliseconds}ms");
sw.Restart();
await locator.InitializeGlobalServicesWithProgress(OnProgress);
Debug.Log($"[Boot] Init: {sw.ElapsedMilliseconds}ms");
#endif
```

## 3.10 Unity Editor Diagnostics Window

The v1 spec extends ServiceLocatorVerification with a ContextMenu. A proper Editor window provides real-time visibility into the registration graph without needing to add ContextMenu calls manually.

New file: Editor/DIGraphWindow.cs

```
public class DIGraphWindow : EditorWindow
```

```

{
    [MenuItem("Core/DI Graph Inspector")]
    public static void Open() => GetWindow<DIGraphWindow>("DI Graph");

    void OnGUI()
    {
        if (!Application.isPlaying) { EditorGUILayout.HelpBox(...); return; }
        var locator = ServiceLocator.Global;
        foreach (var (svcType, desc) in locator.Descriptors)
        {
            var resolved = desc.SingletonInstance != null ? "√" : "o";
            var lifetime = desc.Lifetime.ToString().Substring(0,1);
            EditorGUILayout.LabelField(
                $"{resolved} [{lifetime}] {svcType.Name}",
                $"→ {desc.ImplementationType?.Name ?? "<factory>"}");
        }
    }
}

```

The window displays: resolution status (resolved/pending), lifetime (S/C/T), service interface name, implementation type name. A second tab shows DIValidator output run on demand in Edit Mode. A third tab shows boot timing from the last play session (stored in EditorPrefs).

## 4. Test Infrastructure

The v1 spec describes integration-level verification (ServiceLocatorVerification.cs). No unit test structure is defined. The following test plan covers the container itself, the validator, and the installer sort — these are pure C# and can run in Unity's Test Runner without scene setup.

| Test Class                  | Cases                                                                                                                                                             | Runner         |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>ServiceLocatorTests</b>  | Resolve singleton, resolve transient, detect circular dep, _resolutionStack cleared after exception, thread-safe double-init returns same instance                | EditMode NUnit |
| <b>DIValidatorTests</b>     | Valid graph passes, missing dep throws with type name, [Inject] readonly field is error not warning, static cycle detected, aggregate error count correct         | EditMode NUnit |
| <b>InstallerSorterTests</b> | No deps returns original order, single dep chain sorts correctly, diamond dep sorts correctly, cycle throws, missing dep throws                                   | EditMode NUnit |
| <b>ServiceScopeTests</b>    | Transient returns new instance each call, scoped returns same instance, singleton returns same across scopes, Dispose() calls IDisposable on all scoped instances | EditMode NUnit |

|                                  |                                                                                                                                                                        |                |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>BootstrapIntegrationTests</b> | All installers register cleanly in PlayMode, DIValidator passes after full boot, service resolved in gameplay scene is non-null, scene unload disposes scoped services | PlayMode NUnit |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|

A TestServiceLocator helper (new file: Tests/TestServiceLocator.cs) provides a clean isolated container per test, preventing cross-test contamination of the singleton ServiceLocator.Global:

```
internal static class TestServiceLocator
{
    public static ServiceLocator Create()
    {
        // Returns a new, non-global ServiceLocator instance
        // Tests register into this and discard it after
        return new ServiceLocator(isGlobal: false);
    }
}
```

## 5. Revised Phase Plan

The five original phases are preserved. The table below revises effort estimates to account for the enhancements in §3 and adds a new Phase 0 for test scaffolding and a Phase 6 for final hardening.

| Phase                               | Goal                                                               | Effort (original) | Effort (revised) | Risk   |
|-------------------------------------|--------------------------------------------------------------------|-------------------|------------------|--------|
| <b>Phase 0 — Test Scaffold</b>      | TestServiceLocator, EditMode test projects, CI pipeline wired      | —                 | 0.5 d            | Low    |
| <b>Phase 1 — Critical Bugs</b>      | resolutionStack, IServiceScope cast, Transient, Resources.Load()   | 1 d               | 1.5 d            | Low    |
| <b>Phase 2 — DI Infrastructure</b>  | InstallerSorter, DIValidator, lateInit queue + §3.4 thread safety  | 2 d               | 2.5 d            | Low    |
| <b>Phase 3 — Migrate Installers</b> | All installers → Register<T,TImpl> + §3.3 scope lifecycle          | 2–3 d             | 3 d              | Medium |
| <b>Phase 4 — Structural</b>         | IInstaller async, SaveServiceFactory, open generics (§3.6)         | 2 d               | 2.5 d            | Low    |
| <b>Phase 5 — Boot Wiring</b>        | Bootstrapper validation, perf budget (§3.9), Editor window (§3.10) | 0.5 d             | 1 d              | Low    |
| <b>Phase 6 — Hardening</b>          | IDisposable teardown, scope lifecycle audit, full test pass        | —                 | 1 d              | Low    |

|                               |                                                                                                                                                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Total Revised Estimate</b> | ~12 developer-days across all phases. The additional 4 days (vs. v1's ~8) cover thread-safety hardening, teardown, test infrastructure, and the Editor diagnostics window. Phases 0–2 are low-risk and can be merged immediately. |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6. Updated File Change Reference

All v1 files are preserved. The table below adds only the new or additionally-modified files from this enhancement spec.

| File                                                | Action   | Change Summary                                                                                                                           |
|-----------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------|
| Utility/Attributes/PreferredConstructorAttribute.cs | NEW      | Marks constructor for explicit selection by container and validator                                                                      |
| Systems/Services/ServiceLocator.cs                  | Modify+  | Thread-local _resolutionStack; double-checked singleton lock; Shutdown() with IDisposable; Descriptors public accessor for Editor window |
| Systems/Services/ServiceScope.cs                    | Modify+  | Implement IDisposable; dispose all IDisposable scoped instances on Dispose()                                                             |
| Systems/Services/ServiceDescriptor.cs               | Modify+  | (v1 additions) + no further changes                                                                                                      |
| Systems/Services/InstallerSorter.cs                 | NEW (v1) | v1 spec unchanged                                                                                                                        |
| Systems/Services/DIValidator.cs                     | Modify+  | Static cycle detection (§3.7); [PreferredConstructor] support (§3.1); aggregate error mode (§3.5)                                        |
| Systems/Navigation/SceneServiceScopeRegistry.cs     | Modify+  | Add DisposeForScene() — called by SceneLoader on unload                                                                                  |
| Systems/Loading/Bootstrap/Bootstrapper.cs           | Modify+  | Aggregate installer errors; performance budget logging; call scope.Dispose on scene unload                                               |
| Editor/DIGraphWindow.cs                             | NEW      | Unity Editor window showing registered services, resolution status, boot timing                                                          |
| Tests/EditMode/ServiceLocatorTests.cs               | NEW      | EditMode NUnit unit tests for container                                                                                                  |
| Tests/EditMode/DIValidatorTests.cs                  | NEW      | EditMode NUnit unit tests for validator                                                                                                  |
| Tests/EditMode/InstallerSorterTests.cs              | NEW      | EditMode NUnit unit tests for topological sorter                                                                                         |
| Tests/EditMode/ServiceScopeTests.cs                 | NEW      | EditMode NUnit including IDisposable teardown                                                                                            |
| Tests/PlayMode/BootstrapIntegrationTests.cs         | NEW      | Full boot + scene lifecycle integration test                                                                                             |
| Tests/TestServiceLocator.cs                         | NEW      | Isolated container factory for tests                                                                                                     |

## 7. Risks & Mitigations (Updated)

| Risk                                            | Likelihood    | Impact                               | Mitigation                                                                 |
|-------------------------------------------------|---------------|--------------------------------------|----------------------------------------------------------------------------|
| Register<T,TImpl> misses a ScriptableObject dep | Medium        | <b>Boot error</b>                    | Validator catches it; fallback to lambda registration                      |
| MonoBehaviour Awake fires before InjectServices | Medium        | <b>Null field in Awake</b>           | Move dep-using logic from Awake to Start/OnOpen; add assertion in Awake    |
| Thread race on singleton construction           | Low           | <b>Double-init / data corruption</b> | Double-checked locking on ServiceDescriptor (§3.4)                         |
| IDisposable not called — leaked resources       | High (prefix) | <b>Audio/file handle leak</b>        | ServiceScope.Dispose() added in Phase 6; SceneLoader calls DisposeForScene |
| Circular dep introduced in new service          | Low           | <b>Boot crash</b>                    | Static graph walk in DIValidator catches at boot (§3.7)                    |
| Phase 3 installer regression                    | Medium        | <b>Service null in gameplay</b>      | Dual-registration period + rollback strategy (§3.8) + CI validator         |
| Boot time regression from reflection            | Low           | <b>Load screen slowdown</b>          | Reflection caching + perf budget logging (§3.9)                            |
| Wrong constructor selected silently             | Medium        | <b>Hard dep missing</b>              | [PreferredConstructor] + DIValidator ambiguity warning (§3.1)              |

## 8. Definition of Done

### After Phase 6:

- The container owns the complete dependency graph and validates it at boot before a single frame runs.
- Every service is fully wired at construction — no fields set post-construction except on MonoBehaviours.
- A missing registration produces a named error at startup, not a `NullReferenceException` in gameplay.
- Every service that implements `IDisposable` is disposed when its scope ends — no resource leaks across scene loads.
- Installer ordering is deterministic and enforced by topological sort.
- Circular dependencies are detected statically at boot, not lazily at first resolution.

- Boot time is instrumented; a regression beyond 200 ms throws in development builds.
- All container behaviour is covered by an EditMode NUnit suite that runs in CI on every PR.